Title: Heart Disease Prediction – End-to-End MLOps Pipeline

Author: Patil Sharyu Rupchand

BITS ID: 2025ae05445

Course: Machine Learning Operations (MLOps) AIMLCZG523

Date: October 2026

GitHub Repository: https://github.com/patilsharyurupchanad/mlops

Introduction & Problem Overview

1. Introduction

This project implements a complete MLOps pipeline for Heart Disease Prediction using modern tools including MLflow, FastAPI, Docker, Kubernetes, Prometheus, Grafana, and GitHub Actions.

2. Dataset Overview

- UCI Heart Disease dataset
- 13 clinical features
- Binary classification (0 = No disease, 1 = Disease)

3. Objectives

- Build reproducible ML pipeline
- Track experiments
- Serve model via API
- Containerize and deploy
- Monitor API
- Automate CI/CD
- Produce production-ready documentation

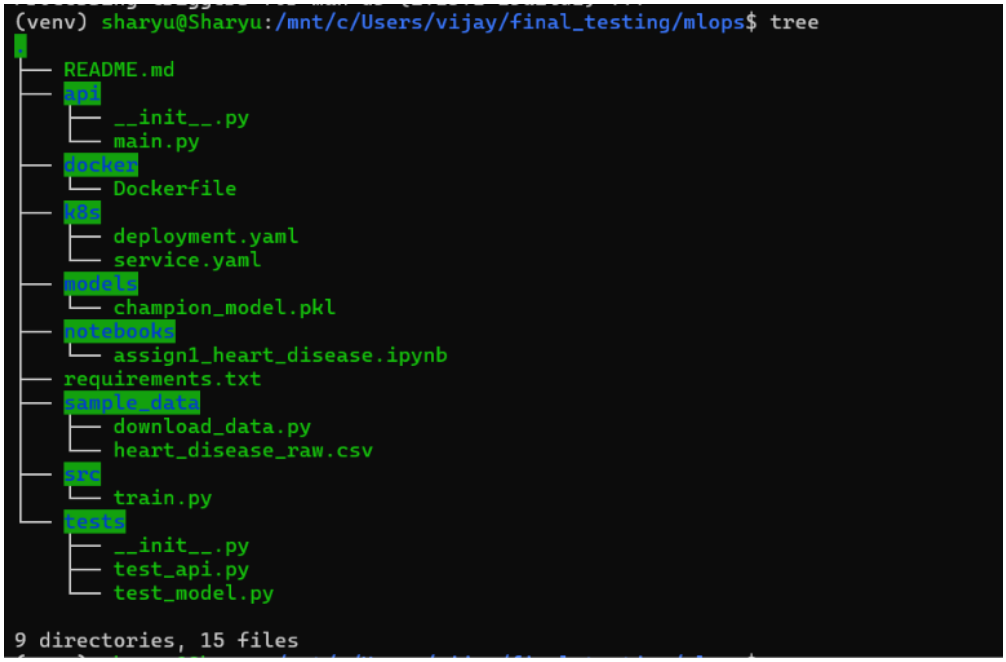
Setup & Installation Instructions

Environment Setup

1. Clone repository
2. Create virtual environment
3. Install dependencies
4. Run notebook

5. Build Docker image
6. Deploy to Kubernetes
7. Enable monitoring
8. Run CI/CD pipeline

Screenshots:



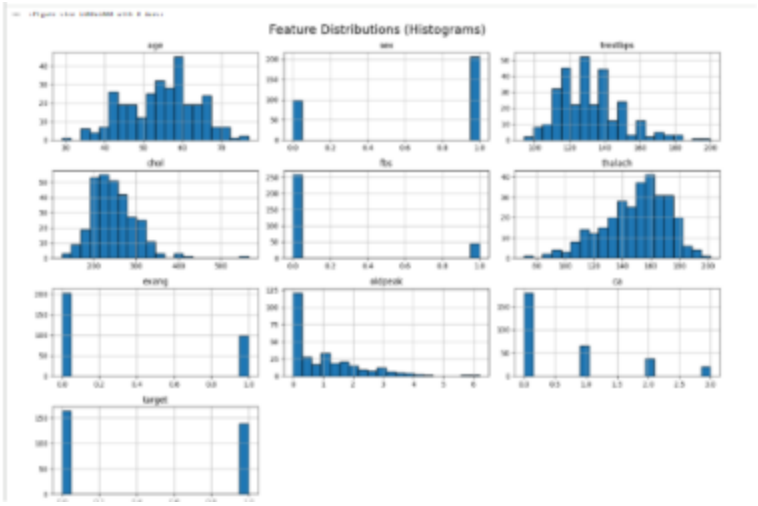
Project folder structure

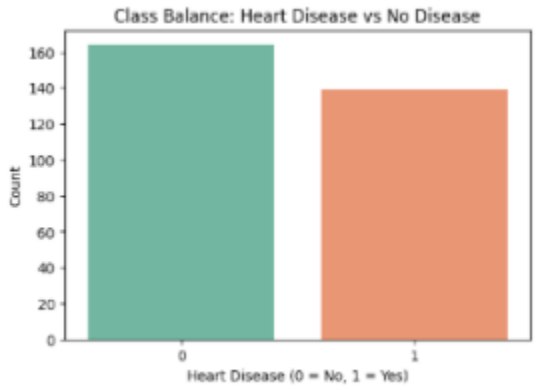
EDA Summary

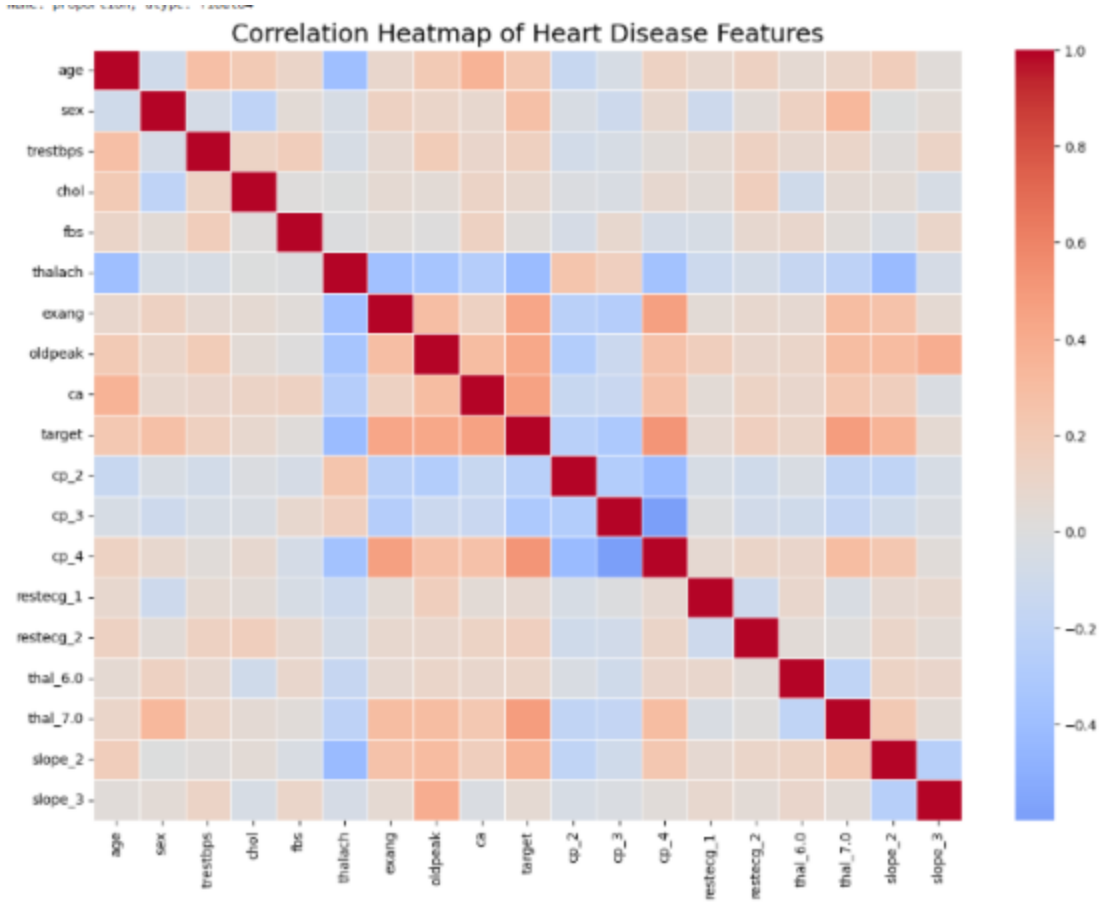
Key Findings

- Age and chest pain type strongly correlated
- Higher resting blood pressure increases risk
- Oldpeak and ST slope are strong predictors
- No missing values after cleaning
- Balanced distribution after preprocessing

Screenshots:


Feature Distributions (Histograms)
age
sex
trestbps
chol
fbs
thalach
exang
oldpeak
ca
target


Class Balance: Heart Disease vs No Disease
Count
0
20
40
60
80
100
120
140
160
0
1
Heart Disease (0 = No, 1 = Yes)



EDA plots (correlation heatmap, distributions).

Modelling Choices

Models Evaluated

- Logistic Regression
- Random Forest
- Gradient Boosting (optional)

Feature Engineering

- One-hot encoding
- Scaling
- Train/test split (80/20)

Champion Model Selection

- MLflow metrics comparison
- Random Forest selected based on highest ROC-AUC

Screenshots:

```
Experiment Name: Heart Disease Detection
Artifact Location: file:///content/MLOps/Heart_Disease_Detection/MLflow/artifacts
Columns containing 'metrics': ['metrics.precision', 'metrics.f1_score', 'metrics.roc_auc', 'metrics.accuracy', 'metrics.recall']
```

	run_id	tags.mlflow.runName	params.model_type	metrics.accuracy	metrics.precision	metrics.recall	metrics.roc_auc	metrics.f1_score
0	3743226488184e9095ae9c7e2ce471d1	Random Forest	Random Forest	0.934426	0.900000	0.964286	0.960498	0.931034
1	5edb815a2df04c54a7a24e3a6d54c0d1	Logistic Regression (skops)	Logistic Regression	0.852459	0.806452	0.892857	0.955628	0.847458
2	c94c372959724f19b834b5b4c65d4750	Random Forest	Random Forest	0.934426	0.900000	0.964286	0.960498	0.931034
3	7afaf4c49da54a8ead006528f5857ad8	Logistic Regression (skops)	Logistic Regression	0.852459	0.806452	0.892857	0.955628	0.847458

```
Experiment name: Heart Disease Detection
Experiment ID: 1
Artifact location: file:///content/MLOps/Heart_Disease_Detection/MLflow/artifacts
MLflow database exists: True
MLflow database path: /content/MLOps/Heart_Disease_Detection/MLflow/mlflow.db
MLflow artifact directory exists: True
MLflow artifact directory: /content/MLOps/Heart_Disease_Detection/MLflow/artifacts

Experiment artifact location:
file:///content/MLOps/Heart_Disease_Detection/MLflow/artifacts
```

MLflow metrics comparison chart.

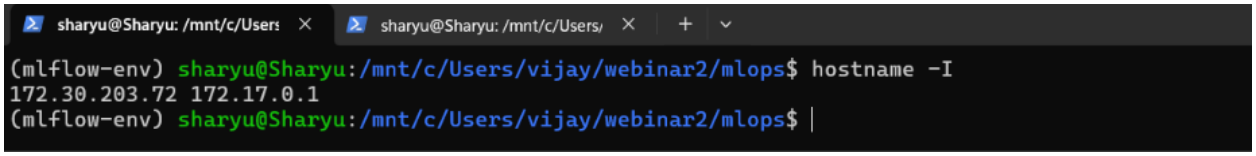
MLflow artifacts (champion_model.pkl).

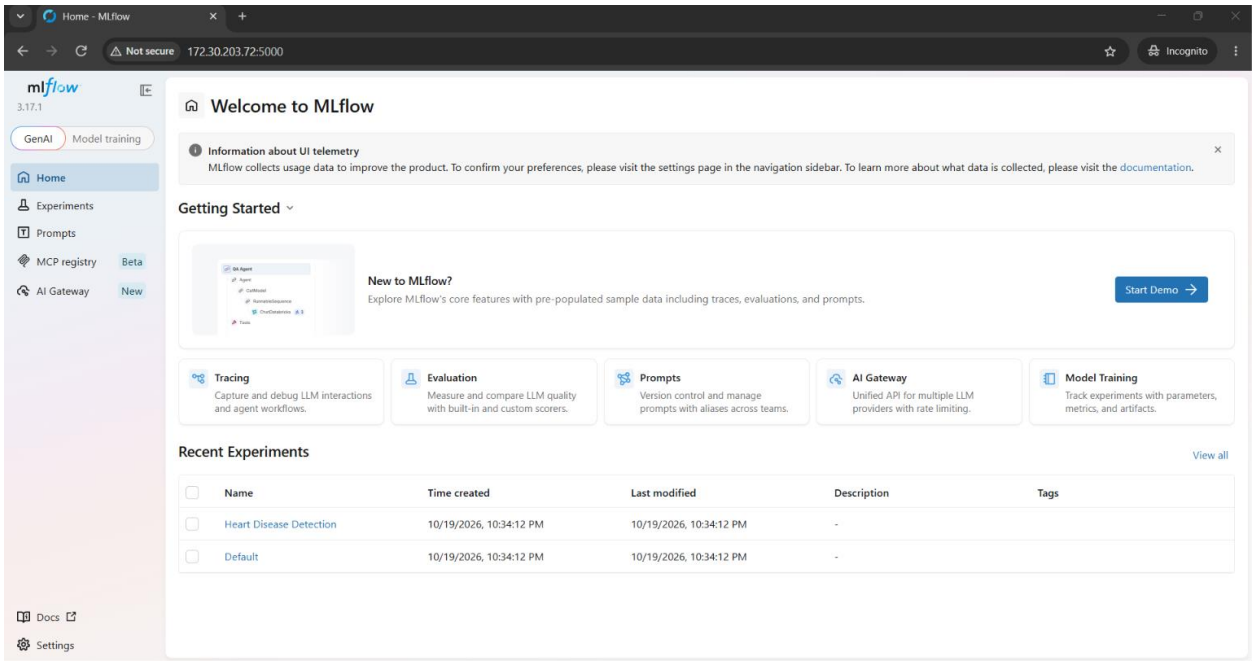
Experiment Tracking (MLflow)

MLflow Tracking

- Parameters logged
- Metrics logged
- Artifacts stored
- Champion model exported
- MLflow_export.zip downloaded automatically

Screenshots:



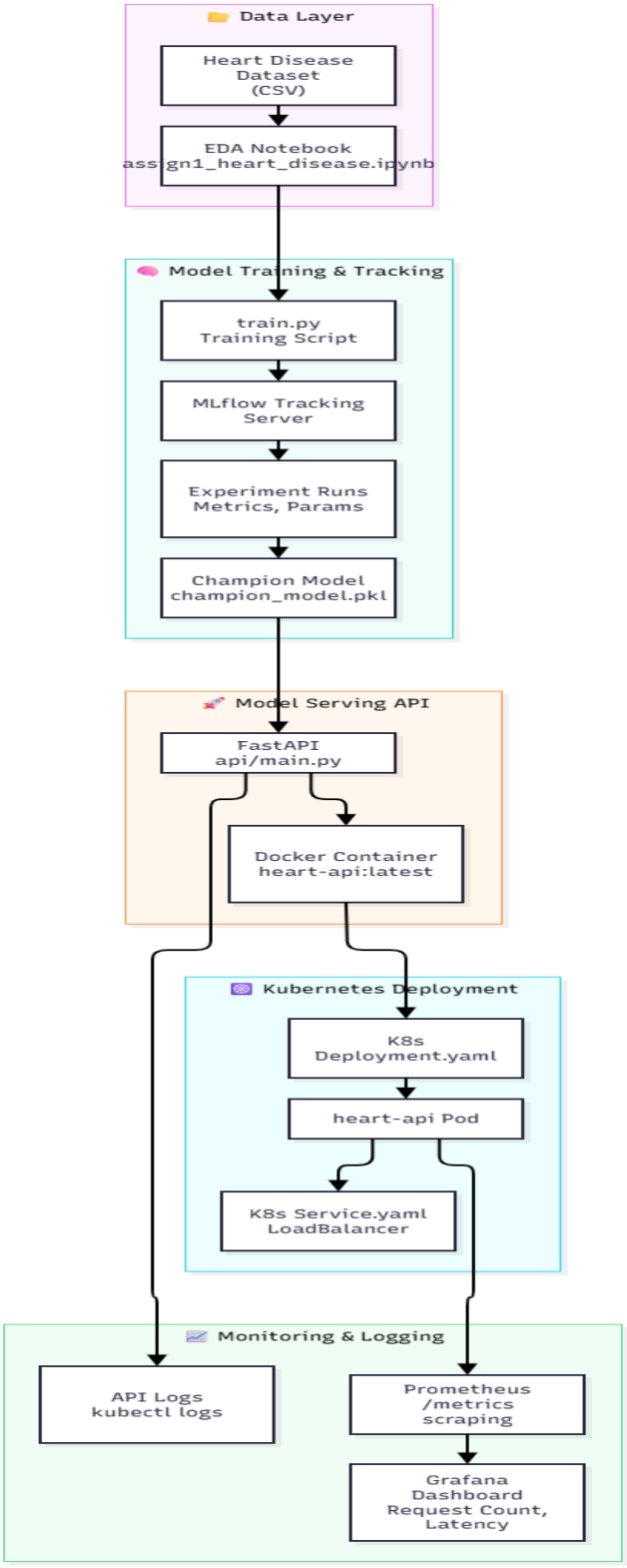


MLflow run details.

Architecture Diagram

System Architecture

Screenshots:



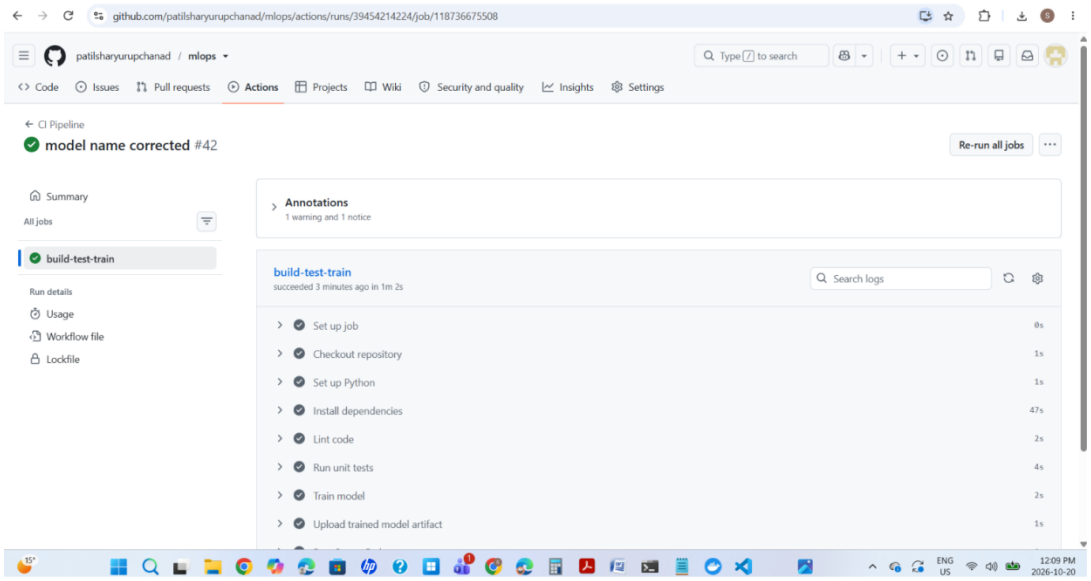
Architecture Diagram (PNG).

CI/CD Pipeline

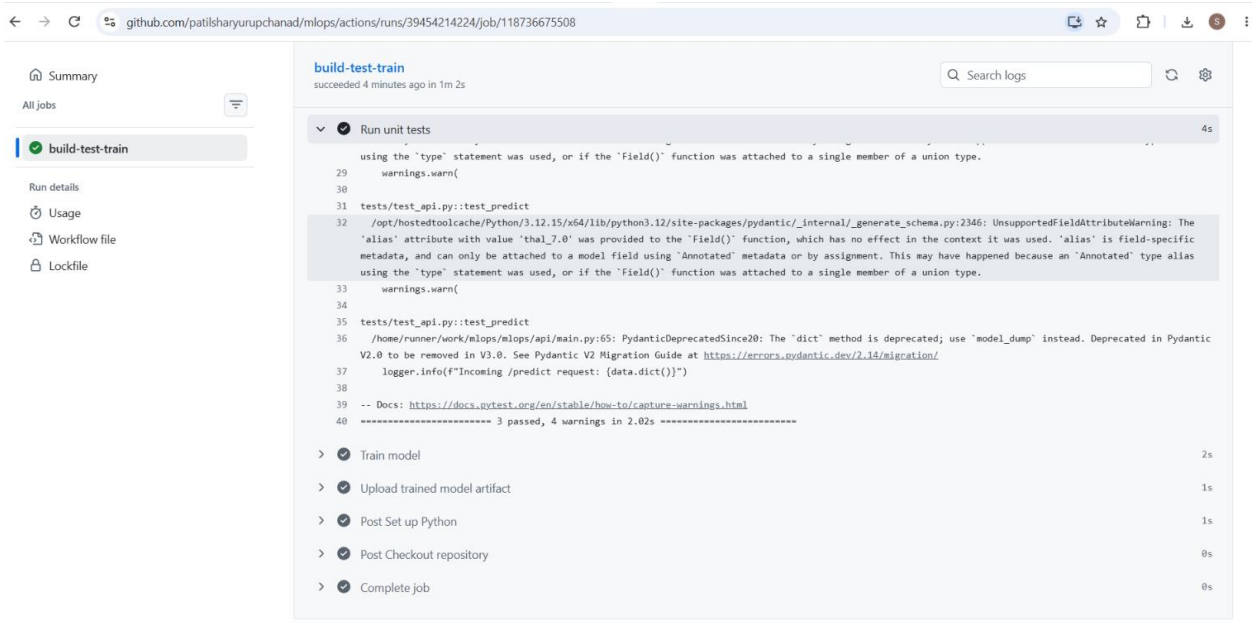
GitHub Actions Workflow

- Linting
- Unit tests
- Model training
- Artifact upload
- Build validation
- Pipeline fails on errors

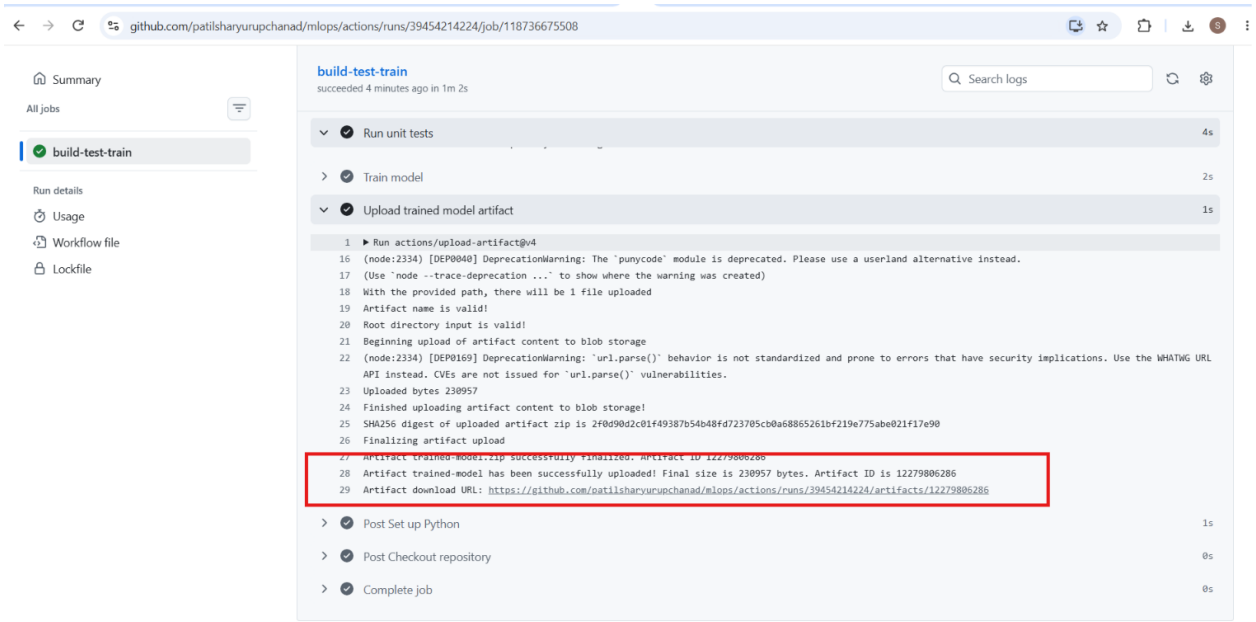
Screenshots:



GitHub Actions pipeline run.



Test results in CI/CD.



Artifact upload in CI/CD.

Deployment (Docker + Kubernetes)

Docker

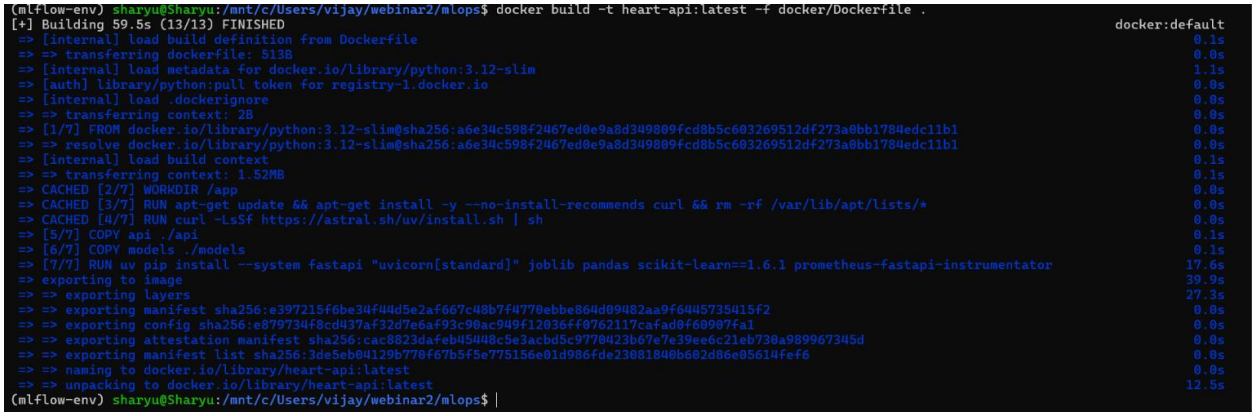
- Built using Dockerfile

- Runs model in isolated environment
- `/health` and `/predict` verified

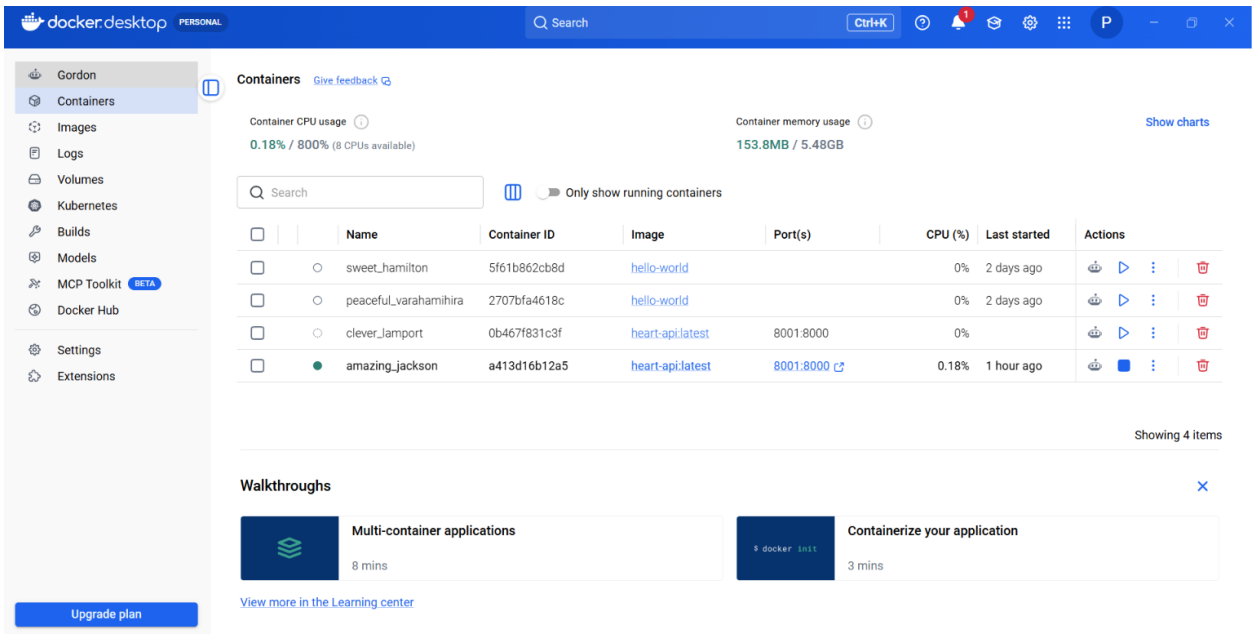
Kubernetes

- Deployment manifest
- LoadBalancer service
- Pod running
- API reachable via port-forward

Screenshots:



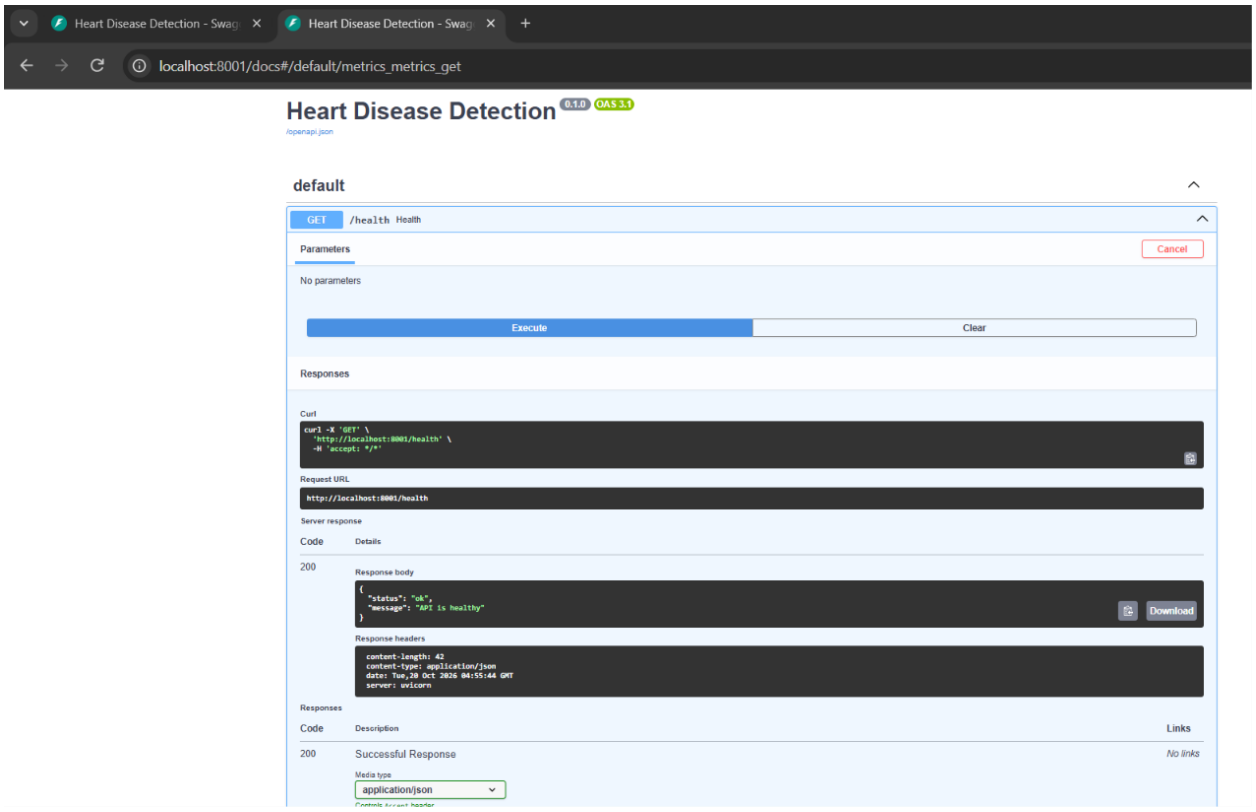
Docker build success.



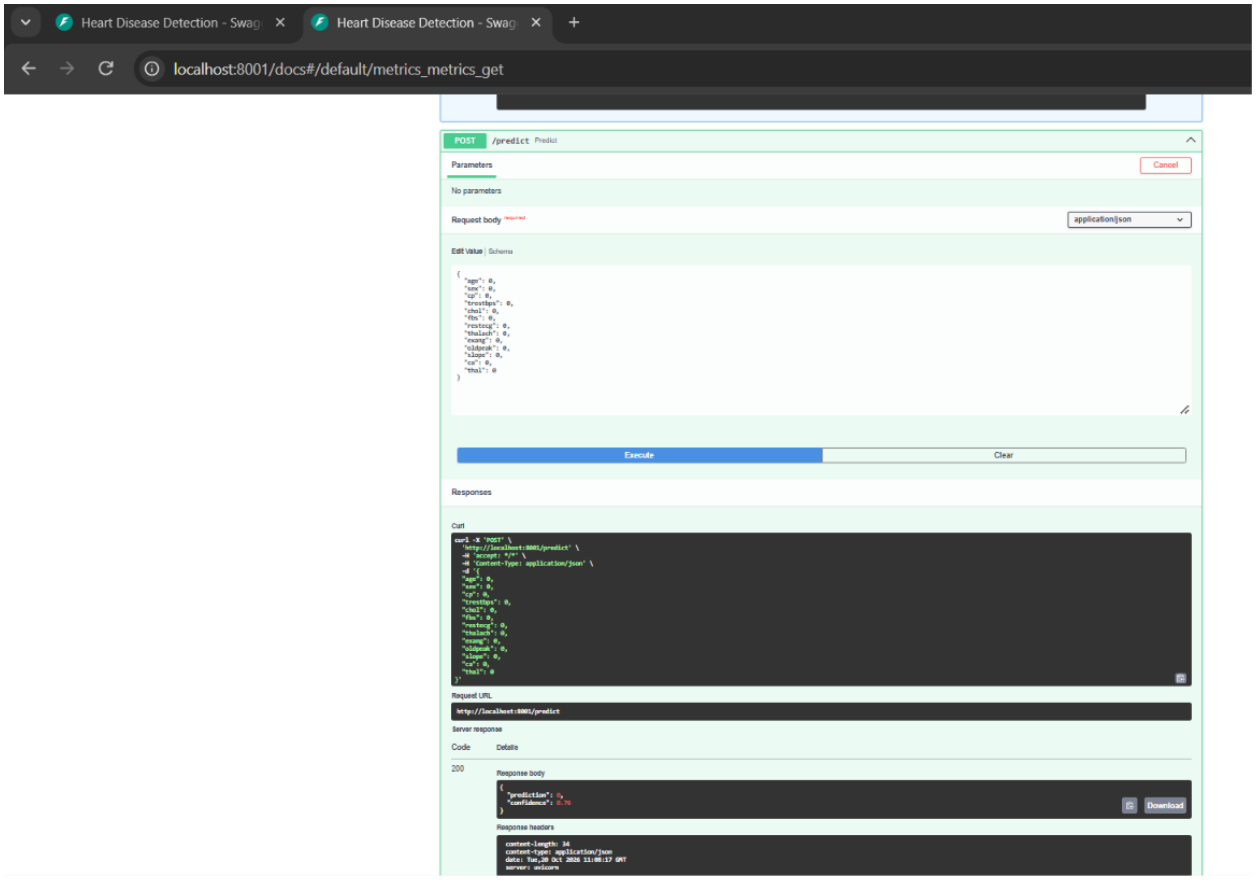
Docker container running.

```
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$ kubectl get pods
NAME                                          READY   STATUS    RESTARTS      AGE
grafana-7fb6698cb6-xkbnn                      1/1     Running   2 (10h ago)   13h
heart-api-deployment-7d9fbfc798-j9kh4         1/1     Running   2 (10h ago)   14h
prometheus-alertmanager-0                     2/2     Running   4 (10h ago)   13h
prometheus-kube-state-metrics-86f6b846bf-zfbqk   1/1     Running   4 (10h ago)   13h
prometheus-prometheus-node-exporter-xmfjc     1/1     Running   2 (10h ago)   13h
prometheus-prometheus-pushgateway-7ffc8d4d8d-lt5pb   1/1     Running   2 (10h ago)   13h
prometheus-server-7c86c64d54-nbn2x            2/2     Running   4 (10h ago)   13h
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$ kubectl get svc
NAME                                TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
grafana                             ClusterIP      10.96.233.48    <none>        80/TCP           13h
heart-api-service                   LoadBalancer   10.96.251.201   172.18.0.4    8000:32676/TCP   16h
kubernetes                          ClusterIP      10.96.0.1       <none>        443/TCP          35h
prometheus-alertmanager             ClusterIP      10.96.15.104    <none>        9093/TCP         13h
prometheus-alertmanager-headless    ClusterIP      None            <none>        9093/TCP         13h
prometheus-kube-state-metrics       ClusterIP      10.96.74.22     <none>        8080/TCP         13h
prometheus-prometheus-node-exporter ClusterIP      10.96.81.41     <none>        9100/TCP         13h
prometheus-prometheus-pushgateway   ClusterIP      10.96.13.36     <none>        9091/TCP         13h
prometheus-server                   ClusterIP      10.96.93.242    <none>        80/TCP           13h
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$ kubectl get deployment
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
grafana                             1/1     1            1           13h
heart-api-deployment                1/1     1            1           16h
prometheus-kube-state-metrics       1/1     1            1           13h
prometheus-prometheus-pushgateway   1/1     1            1           13h
prometheus-server                   1/1     1            1           13h
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$ kubectl config get-contexts
CURRENT   NAME             CLUSTER          AUTHINFO         NAMESPACE
*         docker-desktop   docker-desktop   docker-desktop
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$ kubectl config use-context docker-desktop
Switched to context "docker-desktop".
```

kubectl get pods. And kubectl get svc.



/health endpoint working.



/predict endpoint working.

Monitoring & Logging + Final Notes

Logging

- API logs captured using Python logging
- Logs visible via `kubectl logs`

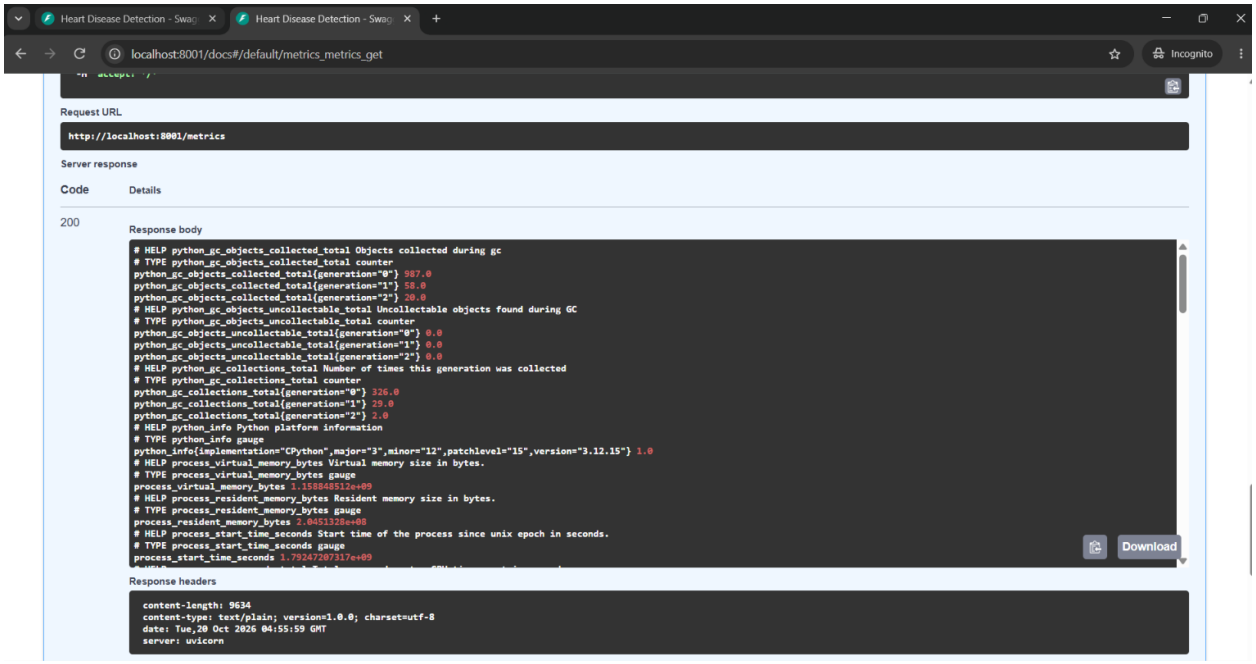
Monitoring

- Prometheus scraping `/metrics`
- Grafana dashboard visualizing request count, latency, predict traffic

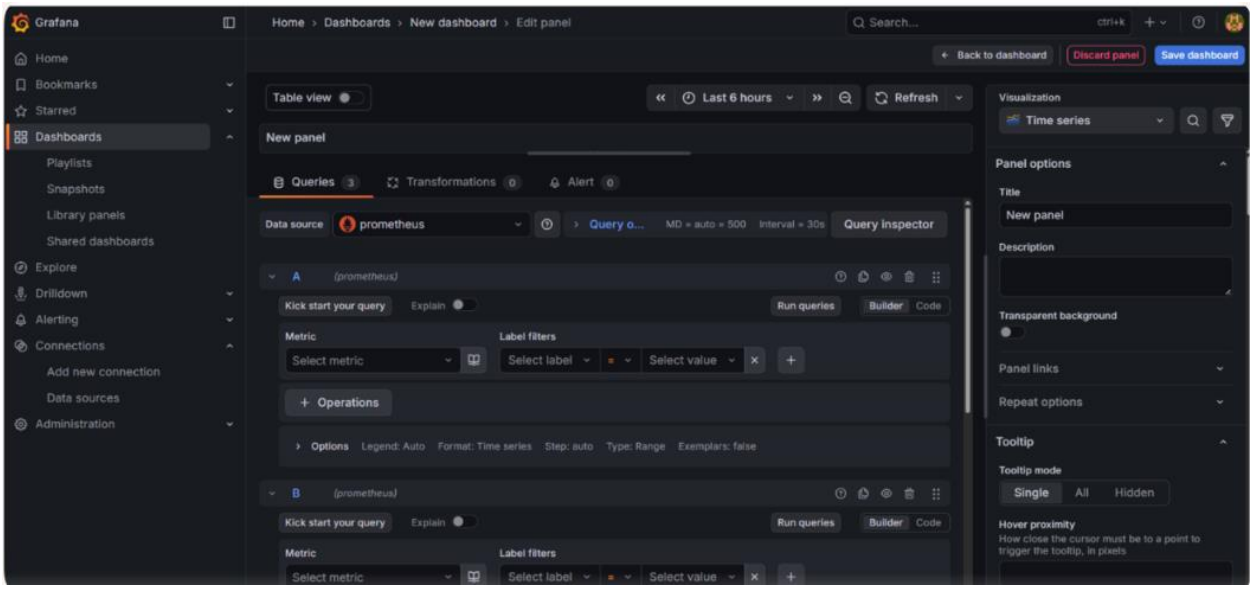
Screenshots:

```
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$ kubectl logs heart-api-deployment-7d9fbfc798-j9kh4
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     10.244.0.1:45756 - "GET /docs HTTP/1.1" 200 OK
INFO:     10.244.0.1:45756 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     10.244.0.1:51864 - "GET /health HTTP/1.1" 200 OK
INFO:     10.244.0.1:57456 - "GET /metrics HTTP/1.1" 200 OK
INFO:     127.0.0.1:44920 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:44920 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:44922 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:49100 - "GET /metrics HTTP/1.1" 200 OK
INFO:     127.0.0.1:56832 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:56832 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:56844 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:56844 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:56844 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:56844 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:58720 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:58720 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:58720 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:58720 - "GET /openapi.json HTTP/1.1" 200 OK
(venv) sharyu@Sharyu:/mnt/c/Users/vijay/final_testing/mlops$
```

API logs from kubectl



/metrics endpoint.



Prometheus target list.

Final Section — Video + Repository Link

Video Demonstration

A short video demonstrating the entire pipeline is included in the repository.

https://drive.google.com/drive/folders/1rJj-qfIuz-OCzpATZE8GQ4NSY71eXc9t?usp=sharing

Repository Link

https://github.com/patilsharyurupchanad/mlops